



UNIVERSIDADE DE BRASÍLIA - UnB

Faculdade de Ciências e Tecnologias em Engenharia – FCTE

Disciplina: Orientação por Objetos

Sistema de Vendas e Fidelidade – Cafeteria Geek "Byte & Brew"

Breno Weber Rabelo

242015129

Gabriel Sakai e Silva

252014223

Julia Susmickat Rizza

241025686

Professor: André Lanna

Turma: 01

Brasília-DF, 29 de Junho de 2026

1. Introdução

Este relatório apresenta o desenvolvimento do Sistema de Vendas e Fidelidade da cafeteria temática Byte & Brew, elaborado como Trabalho final da disciplina de Orientação a Objetos.

1.1 Objetivo

O projeto tem como propósito modelar e implementar uma solução capaz de gerenciar produtos, clientes e vendas, simulando o funcionamento de uma cafeteria e utilizando os princípios fundamentais da Programação Orientada a Objetos (POO).

1.2 Tecnologia Utilizada

Linguagem: Java

Paradigma: Programação Orientada a Objetos (POO)

Bibliotecas: `java.util.ArrayList` e `javax.swing.JOptionPane`.

2. Diagrama de Classes

2.1 Representação Grafica

Link para acessar o diagrama completo:

[UML](#)

3. Pilares de POO Aplicados

3.1 Herança

Obejtivo: Promover o reaproveitamento de código e representar organizar hierarquias.

Benefício: Facilita a manutenção do sistema, melhora a organização das classes e torna o desenvolvimento mais eficiente, permitindo que características comuns sejam compartilhadas e especializadas conforme a necessidade.

Aplicação no sistema:

I) Produto, Comida e Bebida

```
public abstract class Produto
```

```
public class Comida extends Produto
```

```
public class Bebida extends Produto
```

*// A herança foi aplicada entre **Produto**, **Comida** e **Bebida**. A classe **Produto** concentra atributos comuns, como nome, preço, estoque e código. As classes **Comida** e **Bebida** herdam esses atributos e adicionam características próprias.*

II) Cliente, ClienteStandard e ClienteVip

```
public abstract class Cliente
```

```
public class ClienteStandard extends Cliente
```

```
public class ClienteVip extends Cliente
```

// A herança também aparece na hierarquia de clientes.

***ClienteStandard** e **ClienteVip** herdam de **Cliente**, reutilizando nome, CPF, XP e métodos de pontuação.*

3.2 Encapsulamento

Objetivo: Proteger os dados e garantir acesso controlado.

Benefícios: Segurança (evita acesso direto a dados sensíveis) e facilidade de manutenção.

Aplicação no Sistema:

I) Produto

```
private String nomeProduto;  
private double precoBase;  
private int quantidadeEstoque;  
private int codigoProduto;
```

```
public String getNomeProduto() {  
    return nomeProduto;  
}
```

```
public void setPrecoBase(double precoBase) {  
    this.precoBase = precoBase;  
}
```

// Os demais atributos seguem o mesmo padrão de encapsulamento por meio de métodos getters e setters. O encapsulamento é usado para proteger os dados dos produtos. Os atributos são privados e acessados por métodos públicos, evitando alteração direta.

II) Cliente

```
private String nomeCliente, cpf;  
private int xp;
```

```
public String getNome() {  
    return nomeCliente;  
}
```

```
public void setNome(String novoNome) {  
    nomeCliente = novoNome;
```

```
}
```

*// Na classe **Cliente**, dados como **nome**, **CPF** e **XP** são privados, garantindo acesso controlado por getters e setters.*

III) Pedido

```
private ArrayList<ItemPedido> itens;  
private Cliente cliente;  
private int codigoPedido;
```

*// A classe **Pedido** encapsula sua **lista de itens**, **cliente associado** e **código do pedido**, protegendo a estrutura interna da venda.*

3.3 Polimorfismo

Objetivo: Permite que o mesmo método tenha comportamentos diferentes dependendo do objeto que o executa.

Benefícios: Reutilização de código, mais flexível e legível.

Aplicação no Sistema de cada tipo:

3.3.1 Sobrescrita: Acontece quando uma subclasse reescreve um método herdado da superclasse, mantendo o mesmo nome e os mesmos parâmetros.

```
@Override  
public void calcularXP(double valorTotal) {  
    int valorInt;  
    valorInt = (int) valorTotal;  
    adicionarXP(valorInt);  
}
```

```

@Override
public void calcularXP(double valorTotal) {
    int valorInt;
    valorInt = (int) valorTotal;
    adicionarXP(valorInt * 2);
}

```

// O método **calcularXP()** é sobrescrito em **ClienteStandard** e **ClienteVip**. O cliente Standard recebe 1 XP por real gasto, enquanto o VIP recebe o dobro.

3.3.2 Sobrecarga: Ocorre quando existem métodos com o mesmo nome, mas com parâmetros diferentes.

```

public void adicionarItem(Produto produto) throws
    EstoqueInsuficienteException {
    adicionarItem(produto, 1);
}

public void adicionarItem(Produto produto, int
    quantidade) throws EstoqueInsuficienteException {
    ...
}

```

// A sobrecarga ocorre no método **adicionarItem()**, que possui duas assinaturas diferentes: uma para adicionar uma unidade e outra para adicionar uma quantidade específica.

3.3.3 Por inclusão: Acontece quando objetos de subclasses são tratados pelo tipo da superclasse.

```

public void adicionarItem(Produto produto, int quantidade)

private ArrayList<Produto> produtos;

```

*// O sistema permite adicionar tanto **Comida** quanto **Bebida** usando o tipo genérico **Produto**, pois ambas as classes herdam de **Produto**.*

3.3.4 Coerção: Acontece quando há conversão de tipos, de forma explícita ou implícita.

```
int valorInt;  
valorInt = (int) valorTotal;
```

*//A coerção aparece na conversão explícita de **double** para **int** no **cálculo de XP**, pois apenas a parte inteira do valor da compra é usada para pontuação.*

3.3.5 Paramétrico: Ocorre quando usamos estruturas genéricas, como `ArrayList<T>`, em que o mesmo tipo de estrutura pode armazenar diferentes tipos de objetos dependendo do parâmetro informado.

```
private ArrayList<Cliente> clientes;  
  
private ArrayList<Produto> produtos;  
  
private ArrayList<Pedido> pedidos;  
  
private ArrayList<ItemPedido> itens;
```

*// O sistema utiliza **ArrayList<T>**, em que **T** representa o tipo armazenado na lista. Assim, a mesma estrutura **ArrayList** é usada para guardar **clientes**, **produtos**, **pedidos** e **itens de pedido**. Isso caracteriza o uso de generics em Java, ou seja, polimorfismo paramétrico.*

3.4 Abstração

Objetivo: Representar apenas as características e comportamentos essenciais de um objeto, ocultando detalhes de implementação que não são necessários para sua utilização.

Benefícios: Reduz a complexidade do sistema, facilita o reaproveitamento de código, obriga as subclasses a implementarem comportamentos específicos, torna o código mais organizado e de fácil manutenção, e permite trabalhar com conceitos genéricos, deixando os detalhes para as classes especializadas.

Aplicação no sistema:

I) Produto

```
public abstract class Produto {  
    ...  
}
```

*// A classe **Produto** representa um conceito genérico de produto da cafeteria. Ela reúne atributos comuns, como **nome, preço, estoque e código**, mas não pode ser instanciada diretamente. Apenas suas subclasses (**Comida e Bebida**) podem gerar objetos.*

II) Cliente

```
public abstract class Cliente {  
    ...  
}
```

*// A classe **Cliente** representa um cliente de forma genérica, contendo informações comuns, como **nome, CPF e XP**. Ela serve*

de base para as classes **ClienteStandard** e **ClienteVip**, que implementam comportamentos específicos.

III) Método calcularXP()

Método:

```
public abstract void calcularXP(double valorTotal);
```

Implementação em ClienteStandard e ClienteVip:

```
@Override
```

```
public void calcularXP(double valorTotal) {
```

```
    ...
```

```
}
```

```
@Override
```

```
public void calcularXP(double valorTotal) {
```

```
    ...
```

```
}
```

// O método **calcularXP()** é declarado como abstrato na classe **Cliente**, ou seja, não possui implementação. Isso obriga cada subclasse a implementar sua própria lógica de cálculo de XP, de acordo com as regras do programa de fidelidade da cafeteria.

4. Conceitos Complementares da Programação Orientada a Objetos

4.1. Interface:

Objetivo: A interface tem como objetivo definir um contrato que deve ser seguido pelas classes que a implementam.

Benefícios: O uso de interface torna o sistema mais flexível, pois permite criar diferentes tipos de promoções sem alterar a estrutura principal do pedido. Além disso, facilita a manutenção e a expansão do código, já que novas promoções podem ser adicionadas futuramente apenas criando classes que implementem a mesma interface.

Aplicação no Sistema:

// No sistema Byte & Brew, foi criada a interface Promocional, que define o método aplicarDesconto(Pedido pedido).

```
public interface Promocional {  
    double aplicarDesconto(Pedido pedido);  
}
```

// Essa interface é implementada por classes específicas de promoção, como PromocaoEventoGeek e PromocaoFestivalGastronomico.

```
public class PromocaoEventoGeek implements Promocional {  
    ...  
}
```

```
public class PromocaoFestivalGastronomico implements  
Promocional {  
    ...  
}
```

*// A classe **PromocaoEventoGeek** aplica desconto sobre bebidas, enquanto a **PromocaoFestivalGastronomico** aplica desconto sobre comidas. Dessa forma, cada promoção possui sua própria regra, mas todas seguem o mesmo contrato definido pela interface **Promocional**.*

4.2. Modificadores de Acesso:

Objetivo: Controlar a visibilidade dos atributos e métodos das classes, garantindo que os dados sejam acessados e modificados apenas quando necessário.

Benefícios: Protege os dados da classe, evita alterações indevidas nos atributos, melhora o encapsulamento e facilita a manutenção e a organização do código.

Aplicação no sistema:

I) Classe Produto:

```
private String nomeProduto;  
private double precoBase;  
private int quantidadeEstoque;  
private int codigoProduto;
```

```
// atributos da classe Produto foram declarados como  
private, impedindo o acesso direto por outras classes.
```

```
// O acesso e a modificação desses atributos são realizados  
por meio de métodos públicos, como:
```

```
public String getNomeProduto() {  
    return nomeProduto;  
}
```

```
public void setPrecoBase(double precoBase) {
```

```
this.precoBase = precoBase;
```

II) Classe Cliente:

```
private String nomeCliente;  
private String cpf;  
private int xp;
```

*// Os atributos da classe **Cliente** também foram declarados como privados, sendo acessados por métodos públicos.*

```
public String getNome() {  
    return nomeCliente;  
}
```

```
public void setNome(String novoNome) {  
    nomeCliente = novoNome;  
}
```

```
public void setCpf(String novoCpf) {  
    cpf = novoCpf;  
}
```

//Essa abordagem garante maior segurança e controle sobre as informações dos clientes.

4.3. Variáveis Estáticas:

Objetivo: Compartilhar um mesmo valor entre todas as instâncias de uma classe, permitindo controlar informações comuns ao sistema.

Benefícios: Evita duplicação de informações, mantém valores compartilhados entre todos os objetos, facilita o controle de

contadores e regras gerais do sistema, reduz o consumo de memória para dados comuns e aplicação no sistema.

I) Classe Produto:

```
private static int proximoCodigoProduto = 1;
```

```
this.codigoProduto = proximoCodigoProduto++;
```

*// A variável estática **proximoCodigoProduto** é utilizada para gerar automaticamente códigos únicos para cada produto cadastrado.*

II) Classe Pedido:

```
private static int proximoCodigo = 1;
```

*//A variável **proximoCodigo** gera automaticamente um identificador único para cada pedido criado no sistema.*

III) Classe Cliente:

```
private static int realParaXp = 1;
```

```
private static int xpParaReal = 10;
```

// Essas variáveis representam regras compartilhadas do programa de fidelidade da cafeteria.

realParaXp: Define quantos pontos o cliente recebe por real gasto.

xpParaReal: Define a conversão de pontos para desconto nas compras.

Como são variáveis estáticas, esses valores são únicos e compartilhados por todos os clientes do sistema.

5. Associações entre as classes

Tipo	Exemplo	Descrição
Herança	<i>Comida extends Produto</i>	Comida é uma especialização de Produto.
Herança	<i>Bebida extends Produto</i>	Bebida é uma especialização de Produto.
Herança	<i>ClienteStandard extends Cliente</i>	Cliente Standard herda dados e comportamentos de Cliente.
Herança	<i>ClienteVip extends Cliente</i>	Cliente VIP herda de Cliente e possui regras específicas de XP.
Associação	<i>Pedido possui Cliente</i>	Um pedido pode estar associado a um cliente cadastrado.
Composição	<i>Pedido possui ArrayList<ItemPedido></i>	Um pedido é formado por vários itens de pedido.
Associação	<i>ItemPedido possui Produto</i>	Cada item do pedido está ligado a um produto específico.
Dependência	<i>Promocional usa Pedido</i>	A interface de promoção depende de um pedido para aplicar desconto.

6. Exceções Customizadas

Exceção	Justificativa	Onde é usada
<i>NomeInvalidoException</i>	Impede nomes inválidos.	<i>CadastroCliente</i>
<i>CpfInvalidoException</i>	Impede CPF inválido.	<i>CadastroCliente</i>
<i>CpfDuplicadoException</i>	Impede CPF já cadastrado.	<i>CadastroCliente</i>
<i>ClienteNaoEncontradoException</i>	Impede operações em clientes inexistentes.	<i>CadastroCliente</i>
<i>EstoqueInsuficienteException</i>	Impede venda sem estoque suficiente.	<i>Produto / Pedido</i>
<i>PontosInsuficientesException</i>	Impede uso de XP insuficiente.	<i>ClienteVip / Pedido</i>
<i>NenhumProdutoException</i>	Indica ausência de produtos cadastrados.	<i>CadastroProduto</i>
<i>NenhumPedidoException</i>	Indica ausência de pedidos cadastrados.	<i>MenuPedidos / pedidos</i>

Aplicação no sistema:

I) NomeInvalidoException:

```
if (!nomeValido(cliente.getNome())) {  
    throw new NomeInvalidoException();  
}
```

// Essa exceção é lançada quando o nome informado é vazio ou contém caracteres inválidos.

II) **CpfInvalidoException:**

```
} catch (NomeInvalidoException | CpfInvalidoException |  
CpfDuplicadoException e) {
```

```
JOptionPane.showMessageDialog(null, e.getMessage());  
}
```

//Essa exceção impede o cadastro de clientes com CPF fora do formato esperado de 11 números.

III) **CpfDuplicadoException:**

```
if (cpfJaExiste(cliente.getCpf())) {  
    throw new CpfDuplicadoException();  
}
```

// Essa exceção evita que dois clientes sejam cadastrados com o mesmo CPF.

IV) **ClienteNaoEncontradoException:**

```
throw new ClienteNaoEncontradoException();
```

// É usada quando uma busca por CPF não encontra nenhum cliente cadastrado.

V) EstoqueInsuficienteException:

```
if (quantidade > quantidadeEstoque) {  
    throw new EstoqueInsuficienteException(...);  
}
```

// Essa exceção impede que o sistema aceite pedidos com quantidade maior que o estoque disponível.

VI) PontosInsuficientesException :

```
if (getXp() < xpNecessario) {  
    throw new PontosInsuficientesException();  
}
```

// Essa exceção impede que o Cliente VIP pague com XP quando não possui pontos suficientes.

VII) NenhumProdutoException:

```
if (produtos.isEmpty()) {  
    throw new NenhumProdutoException();  
}
```

// Essa exceção é lançada quando não existem produtos cadastrados no sistema.

VIII) NenhumPedidoException:

```
if (pedidos.isEmpty()) {  
    throw new NenhumPedidoException();  
}
```

// Essa exceção é lançada quando não existem pedidos cadastrados no sistema.